**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**

**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА**

**Факультет електроніки та комп'ютерних технологій**

# Звіт

про виконання лабораторної роботи №4

"Інженерія даних для задачі класифікації"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

ас. Фостяк М. Р.

**Львів-2026**

**Мета роботи:** Навчитися виконувати підготовку та перетворення даних для задачі класифікації (Automobile_data.csv), формувати різні набори ознак (повний, відібрані за Feature Importance, PCA) та досліджувати вплив інженерії даних на якість моделі машинного навчання на основі набору даних автомобілів.

**Теоретичні відомості:**

Інженерія даних для класифікації включає комплекс методів підготовки ознак: очищення даних та імпутацію пропусків (медіаною для числових та модою для категорійних ознак), стандартизацію числових параметрів (StandardScaler) та унітарне кодування категорій (OneHotEncoder). Для оптимізації ознакового простору застосовують відбір найбільш інформативних ознак (Feature Importance) та метод головних компонент (PCA). Критичною вимогою є запобігання витоку даних (Data Leakage): усі параметри перетворень навчаються виключно на train-вибірці, а оцінка якості здійснюється на незалежній test-вибірці за метриками Accuracy, Precision, Recall, F1-score, ROC-AUC та Confusion Matrix.

## Виконання роботи

1. Завдання 1. Підготовка даних за результатами розвідувального аналізу (EDA): Зчитав датасет Automobile_data.csv (205 рядків, 26 колонок) з UCI ML. Дублікатів 0. Символи '?' замінив на np.nan, числові стовпці привів до float. Пропуски виявлено у 7 стовпцях (6 числових, 1 категоріальний). Імпутація: числові поля заповнив стійкими медіанами, num-of-doors - модою ('four'). Ціль symboling бінаризував: y = 1 (ризикові, > 0) та y = 0 (безпечні, <= 0). Матрицю X сформував із 25 ознак. Витоків інформації у даних немає.

```
# зчитування, обробка пропусків та визначення цілі y
df = pd.read_csv('Automobile_data.csv').replace('?', np.nan)
num_cols = ['wheel-base', 'length', 'width', 'height', 'curb-weight', 'engine-size', 'bore',
            'stroke', 'compression-ratio', 'horsepower', 'peak-rpm', 'city-mpg', 'price', 'normalized-losses']
cat_cols = ['make', 'fuel-type', 'aspiration', 'num-of-doors', 'body-style', 'drive-wheels', 'fuel-system']
for c in num_cols: df[c] = pd.to_numeric(df[c], errors='coerce').fillna(df[c].median())
for c in cat_cols: df[c] = df[c].fillna(df[c].mode()[0])
X, y = df[num_cols + cat_cols], (df['symboling'] > 0).astype(int)
```

У таблиці 1 наведено зведені результати аналізу пропусків та параметри імпутації:

| Ознака (Feature) | Тип даних | Кількість пропусків | Частка (%) | Метод імпутації | Значення заповнення |
|---|---|---|---|---|---|
| normalized-losses | Числова (float) | 41 | 20.00% | Медіана (median) | 115.0 |
| bore | Числова (float) | 4 | 1.95% | Медіана (median) | 3.31 |
| stroke | Числова (float) | 4 | 1.95% | Медіана (median) | 3.29 |
| horsepower | Числова (float) | 2 | 0.98% | Медіана (median) | 95.0 |
| peak-rpm | Числова (float) | 2 | 0.98% | Медіана (median) | 5200.0 |
| price | Числова (float) | 4 | 1.95% | Медіана (median) | 10295.0 |
| num-of-doors | Категоріальна (object) | 2 | 0.98% | Мода (mode) | 'four' (чотиридверні) |

2. Завдання 2. Формування навчальної та тестової вибірок (80% / 20%):
Розділив вихідні дані на тренувальну (164 екземпляри, 80%) та тестову (41 екземпляр, 20%) вибірки за допомогою функції train_test_split з фіксацією random_state=42. Обов'язково встановив параметр stratify=y для збереження природної пропорції класів в обох вибірках.
Правило запобігання витоку даних (Data Leakage): усі наступні кроки налаштування параметрів (розрахунок середнього і дисперсії для StandardScaler, виділення категорій для OneHotEncoder, оцінка важливості ознак та проекційні осі PCA) виконуються виключно на train-вибірці (.fit() / .fit_transform()). Тестова вибірка залишається строго ізольованою аж до моменту фінального тестування (.transform()).

```
# стратифікований поділ на train та test 80/20
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

3. Завдання 3. Підготовка числових і категоріальних ознак через ColumnTransformer:
Сформував конвеєр перетворення ознак за допомогою ColumnTransformer:
- До 15 числових ознак застосував StandardScaler (центрування навколо нуля та нормалізація дисперсії до 1);
- До 10 категоріальних ознак застосував OneHotEncoder(handle_unknown='ignore', sparse_output=False), що згенерувало 58 бінарних колонок.
Після об'єднання вихідні 25 колонок перетворено у 73 стандартизовані числові ознаки (15 числових + 58 One-Hot).

```
# пайплайн обробки ознак ColumnTransformer
preprocessor = ColumnTransformer(transformers=[
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), cat_cols)
])
X_train_prep = preprocessor.fit_transform(X_train)
X_test_prep = preprocessor.transform(X_test)
feature_names = preprocessor.get_feature_names_out()
```

```
initial features count: 25
preprocessed features count: 73
class distribution in train:
 symboling
 1    90
 0    74
 Name: count, dtype: int64
```

4. Завдання 4. Аналіз дисбалансу цільової змінної:
Дослідив частотний розподіл класів цільової змінної symboling у повному датасеті, train- та test-вибірках (таблиця 2).

```
# аналіз розподілу класів цільової змінної
print('train class distribution:\n', y_train.value_counts(normalize=True))
print('test class distribution:\n', y_test.value_counts(normalize=True))
```

| Вибірка | Клас 0 (<= 0, Безпечні) | Частка 0 (%) | Клас 1 (> 0, Ризикові) | Частка 1 (%) | Загалом |
|---|---|---|---|---|---|
| Повний датасет (Full) | 92 | 44.88% | 113 | 55.12% | 205 |
| Навчальна вибірка (Train) | 74 | 45.12% | 90 | 54.88% | 164 |
| Тестова вибірка (Test) | 18 | 43.90% | 23 | 56.10% | 41 |

Обґрунтування балансування: у вибірці клас 0 становить 44.88% (92 авто), а клас 1 становить 55.12% (113 авто) (співвідношення 1:1.23, помірний природний баланс). Застосування SMOTE чи Random Undersampling є недоцільним: SMOTE у розрідженому просторі після One-Hot кодування схильний створювати шумні штучні точки, а ансамбль дерев Random Forest природно стійкий до такої легкої асиметрії частот. Стратифікований поділ надійно зберіг точні частки класів у навчальній та тестовій вибірках.

5. Завдання 5. Формування трьох наборів ознак (Набір A, Набір B, Набір C): Сформував три альтернативні простори представлення ознак:
- Набір A (всі ознаки): повний вектор із 73 стандартизованих ознак;
- Набір B (відібрані ознаки): ТОП-15 найбільш важливих ознак за показником Feature Importance моделі Random Forest;
- Набір C (PCA 90% дисперсії): зниження розмірності методом головних компонент (PCA). Навчив PCA на X_train_prep і відібрав кількість компонент для покриття >= 90% сумарної дисперсії.

```
# набір b (топ-15 ознак) та набір c (pca 90% дисперсії)
importances = clf_a.feature_importances_
top_idx = np.argsort(importances)[::-1][:15]
X_train_b, X_test_b = X_train_prep[:, top_idx], X_test_prep[:, top_idx]

pca = PCA(n_components=0.90, random_state=42)
X_train_c = pca.fit_transform(X_train_prep)
X_test_c = pca.transform(X_test_prep)
```

У таблиці 3 наведено ТОП-15 відібраних ознак для Набору B із зазначенням їх важливості та фізичного змісту:

| Ранг | Назва ознаки (Feature) | Важливість | Тип | Фізичний зміст та вплив на ризик |
|---|---|---|---|---|
| 1 | num-of-doors_two | 0.1594 | OHE | Дводверні купе/спорткари мають суттєво вищий ризик аварійності |
| 2 | num-of-doors_four | 0.1071 | OHE | Чотиридверні сімейні седани вирізняються нижчим ризиком |
| 3 | wheel-base | 0.0764 | Числова | Колісна база: коротка база властива маневреним спортивним авто |
| 4 | normalized-losses | 0.0741 | Числова | Нормалізовані страхові збитки: прямий індикатор аварійності |
| 5 | height | 0.0565 | Числова | Висота кузова: низький профіль типовий для спортивних автомобілів |

| | | | | |
|---|---|---|---|---|
| 6 | length | 0.0523 | Числова | Габаритна довжина: компактні авто частіше обирають активні водії |
| 7 | curb-weight | 0.0492 | Числова | Споряджена маса: легкі авто частіше зазнають пошкоджень |
| 8 | width | 0.0452 | Числова | Ширина кузова автомобіля |
| 9 | engine-size | 0.0329 | Числова | Робочий об'єм двигуна: висока літражність провокує динамічну їзду |
| 10 | peak-rpm | 0.0313 | Числова | Максимальні оберти: показник форсованих високооборотних моторів |
| 11 | horsepower | 0.0289 | Числова | Потужність двигуна у к.с.: прямий корелят високої швидкості та ризику |
| 12 | price | 0.0269 | Числова | Ринкова вартість: дорогі спорткари мають специфічний ризик |
| 13 | compression-ratio | 0.0248 | Числова | Ступінь стиснення двигуна (тип пального/турбонаддув) |
| 14 | bore | 0.0241 | Числова | Діаметр циліндра двигуна |
| 15 | body-style_hatchback | 0.0201 | OHE | Кузов хетчбек: масовий молодіжний міський сегмент авто |

Для Набору C за методом PCA знадобилося 13 головних компонент (PC1 - PC13), які пояснюють 90.63% повної дисперсії (таблиця 4):

| Комп. | Дисперсія | Частка | Накопич. | Комп. | Дисперсія | Частка | Накопич. |
|---|---|---|---|---|---|---|---|
| PC1 | 0.4166 | 41.7% | 41.7% | PC8 | 0.0239 | 2.4% | 83.1% |
| PC2 | 0.1497 | 15.0% | 56.6% | PC9 | 0.0191 | 1.9% | 85.0% |
| PC3 | 0.0767 | 7.7% | 64.3% | PC10 | 0.0172 | 1.7% | 86.8% |
| PC4 | 0.0527 | 5.3% | 69.6% | PC11 | 0.0150 | 1.5% | 88.3% |
| PC5 | 0.0455 | 4.5% | 74.1% | PC12 | 0.0126 | 1.3% | 89.5% |
| PC6 | 0.0369 | 3.7% | 77.8% | PC13 | 0.0110 | 1.1% | 90.6% |
| PC7 | 0.0295 | 2.9% | 80.7% | - | - | - | - |

6. Завдання 6. Побудова моделі класифікації для всіх експериментів:
Для об'єктивного дослідження впливу простору ознак використав єдиний алгоритм з фіксованими параметрами: RandomForestClassifier(n_estimators=100, random_state=42). Фіксація моделі гарантує, що зміни якості зумовлені виключно представленням даних:
- Model A: навчання на всіх 73 ознаках (X_train_prep);
- Model B: навчання на 15 відібраних ознаках (X_train_b);
- Model C: навчання на 13 головних компонентах PCA (X_train_c).

```
# навчання трьох моделей random forest з n_estimators=100
clf_a = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_train_prep, y_train)
clf_b = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_train_b, y_train)
clf_c = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_train_c, y_train)

y_pred_a, y_prob_a = clf_a.predict(X_test_prep), clf_a.predict_proba(X_test_prep)[:, 1]
y_pred_b, y_prob_b = clf_b.predict(X_test_b), clf_b.predict_proba(X_test_b)[:, 1]
y_pred_c, y_prob_c = clf_c.predict(X_test_c), clf_c.predict_proba(X_test_c)[:, 1]
```

7. Завдання 7. Оцінювання моделей та побудова матриць невідповідностей:
Для кожної моделі отримав прогнози на контрольній тестовій вибірці (41 авто: 18 безпечних, 23 ризикових). Розрахував Accuracy, Precision, Recall, F1-score, ROC-AUC та матриці невідповідностей (TN, FP, FN, TP) (таблиця 5):

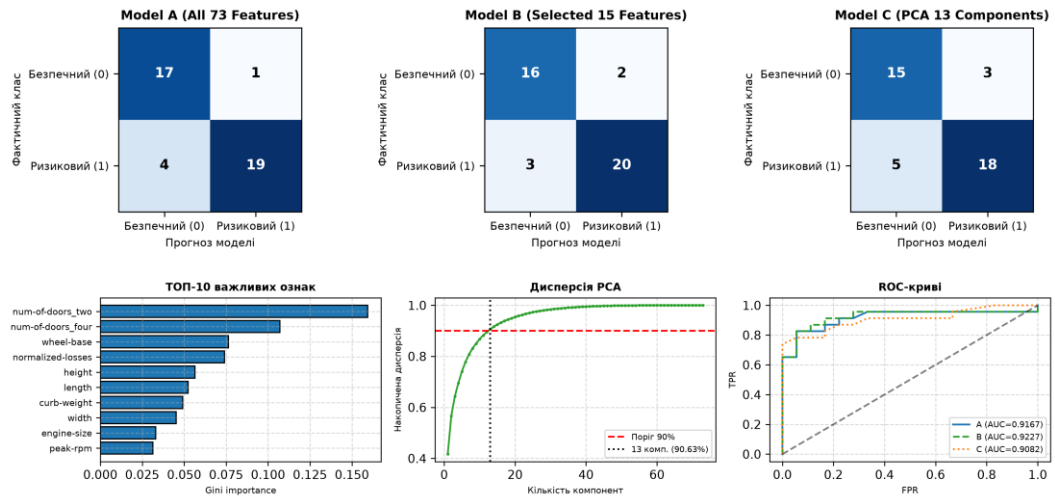
```python
# розрахунок матриць невідповідностей та підсумкових метрик
cm_a, cm_b, cm_c = confusion_matrix(y_test, y_pred_a), confusion_matrix(y_test, y_pred_b), confusion_matrix(y_test, y_pred_c)
metrics = []
for name, n_feat, pred, prob in [('Model A (All 73)', 73, y_pred_a, y_prob_a),
                                 ('Model B (Top-15)', 15, y_pred_b, y_prob_b),
                                 ('Model C (PCA 13)', 13, y_pred_c, y_prob_c)]:
    metrics.append({
        'Model': name, 'Features': n_feat,
        'Accuracy': round(accuracy_score(y_test, pred), 4),
        'Precision': round(precision_score(y_test, pred), 4),
        'Recall': round(recall_score(y_test, pred), 4),
        'F1-score': round(f1_score(y_test, pred), 4),
        'ROC-AUC': round(roc_auc_score(y_test, prob), 4)
    })
print(pd.DataFrame(metrics).to_string(index=False))
```

| Модель | Ознаки | Accuracy | Precision | Recall | F1-score | ROC-AUC | TN | FP | FN | TP |
|---|---|---|---|---|---|---|---|---|---|---|
| Model A (All 73) | 73 | 0.8780 | 0.9500 | 0.8261 | 0.8837 | 0.9167 | 17 | 1 | 4 | 19 |
| Model B (Selected 15) | 15 | 0.8780 | 0.9091 | 0.8696 | 0.8889 | 0.9227 | 16 | 2 | 3 | 20 |
| Model C (PCA 13) | 13 | 0.8049 | 0.8571 | 0.7826 | 0.8182 | 0.9082 | 15 | 3 | 5 | 18 |

Нижче наведено програмний код побудови візуалізацій та сформовані порівняльні графіки результатів моделювання:
- Рисунок 1: зіставлення трьох матриць невідповідностей (Confusion Matrices) для Model A, Model B та Model C;
- Рисунок 2: діаграма ТОП-10 ознак, крива накопиченої дисперсії PCA з порогом 90% та зіставлення ROC-кривих.

```python
# візуалізація матриць невідповідностей та підсумкових графіків
fig1, axes1 = plt.subplots(1, 3, figsize=(14, 4.2))
for ax, (t, cm) in zip(axes1, [('Model A', cm_a), ('Model B', cm_b), ('Model C', cm_c)]):
    ax.imshow(cm, cmap='Blues')
    for i in range(2):
        for j in range(2): ax.text(j, i, str(cm[i, j]), ha='center', va='center')

fig2, axes2 = plt.subplots(1, 3, figsize=(16, 4.5))
axes2[0].barh(range(10), importances[top10_idx][::-1], color='#1f77b4')
axes2[1].plot(range(1, 74), np.cumsum(PCA().fit(X_train_prep).explained_variance_ratio_))
axes2[2].plot(fpr_a, tpr_a, label='Model A'); axes2[2].plot(fpr_b, tpr_b, label='Model B')
axes2[2].plot(fpr_c, tpr_c, label='Model C'); axes2[2].legend()
```




8. Завдання 8. Детальний порівняльний аналіз результатів (відповіді на 7 запитань методички):
1. Чи покращив відбір ознак якість класифікації?
Так, відбір 15 найбільш важливих ознак покращив ключові метрики якості. Повнота виявлення ризикових авто (Recall) зросла з 0.8261 до 0.8696 (+4.35%), узагальнена F1-score підвищилася з 0.8837 до 0.8889, а ROC-AUC зріс з 0.9167 до 0.9227 при збереженні Accuracy = 0.8780. Кількість критичних помилок другого

роду (FN) знизилася з 4 до 3.
2. Як змінилася точність після зменшення кількості ознак?
Загальна точність (Accuracy) збереглася на рівні 0.8780 (87.80%), але баланс помилок суттєво покращився: модель перестала перенавчатися на рідкісних категоріях One-Hot кодування і стала чутливішою до ризикових автомобілів.
3. Скільки ознак можна вилучити без суттєвого погіршення результату?
Без жодної втрати якості можна впевнено вилучити 58 ознак із 73 (майже 80% простору). Модель B на 15 ознаках показала якість вищу за Recall та ROC-AUC, ніж модель на всіх 73 ознаках. Це доводить, що 58 відкинутих ознак були надлишковим шумом.
4. Яку частку дисперсії пояснюють вибрані PCA-компоненти?
Вибрані 13 головних компонент сумарно пояснюють 90.63% повної дисперсії стандартизованих даних. Перша компонента PC1 концентрує 41.66% дисперсії, а друга PC2 - 14.97% (разом перші дві покривають 56.63% усієї варіативності).
5. Як PCA вплинув на Accuracy, F1 та ROC-AUC?
Застосування PCA суттєво погіршило всі метрики: Accuracy впала з 0.8780 до 0.8049 (-7.31%), F1-score знизився з 0.8889 до 0.8182 (-7.07%), Recall впав до 0.7826, а ROC-AUC зменшився з 0.9227 до 0.9082. Кількість помилок зросла до 8 (3 FP та 5 FN).
6. Чи виправдане використання PCA для цього набору даних?
Використання PCA є невиправданим з трьох причин: 1) дерева рішень будують розбиття паралельно координатним осям, а лінійні суміші ознак PCA розмивають пороги; 2) втрачається фізична інтерпретованість ознак; 3) прямий відбір ознак показав вищу якість.
7. Який варіант представлення даних забезпечує найкраще співвідношення між якістю моделі та складністю даних?
Найкращим варіантом є Набір B (ТОП-15 ознак за Feature Importance). Він досягає максимального F1-score (0.8889), найвищого ROC-AUC (0.9227) та найвищого Recall (0.8696) за мінімуму помилок FN=3, скорочує розмірність у 5 разів і зберігає інтерпретованість.

9. Завдання 9. Побудова фінального pipeline класифікації:

На основі проведених експериментів найкращим обрано пайплайн на базі Набору B (попередня обробка + відбір ТОП-15 ознак + Random Forest).
Послідовність етапів: Raw Data -> Data Cleaning -> Stratified Train/Test Split (80/20) -> ColumnTransformer (StandardScaler + OneHotEncoder) -> Feature Selection (Top-15 Importances) -> RandomForestClassifier -> Prediction & Evaluation.
Нижче наведено зведену таблицю архітектури фінального pipeline з детальним описом функцій кожного вузла та режиму навчання для гарантії унеможливлення витоку даних (Data Leakage).

| Етап (Stage) | Компонент / Трансформер | Дія на Train (X_train) | Дія на Test (X_test) | Захист від Data Leakage |
|---|---|---|---|---|
| 1. Ingestion & Cleaning | Pandas fillna (median/mode) | Розрахунок медіан числових та мод категорійних ознак | Заповнення заздалегідь розрахованими train-значеннями | Параметри імпутації не бачать тестову вибірку |
| 2. Dataset Splitting | train_test_split (stratify=y) | Формування ізольованої train-вибірки 80% (164 рядки) | Повна ізоляція тестової вибірки 20% (41 рядок) | Розбиття виконується до будь-яких статистичних перетворень |
| 3. Preprocessing | ColumnTransformer (Scaler + OHE) | .fit_transform(X_train) - розрахунок mean/std та категорій | .transform(X_test) - застосування отриманих параметрів | Тестові спостереження не впливають на нормалізацію та кодування |
| 4. Feature Selection | Top-15 Feature Importances | .fit(X_train_prep, y_train) та визначення топ-15 індексів | Зріз матриці за тими самими індексами: X_test[:, top_idx] | Відбір ознак за тестовою вибіркою був би грубим витоком |
| 5. Classifier Model | RandomForestClassifier(100) | .fit(X_train_b, y_train) - навчання ансамблю з 100 дерев | .predict() та .predict_proba() | Ваги та структура дерев базуються виключно на train |
| 6. Evaluation | Accuracy, Precision, Recall, F1, ROC | Контроль перенавчання (train score) | Фінальна незалежна оцінка узагальнюючої здатності | Чесна оцінка без оптимістичного зміщення метрик |

## AI Black Box

У цій роботі я використовував Gemini для проєктування ColumnTransformer зі StandardScaler та OneHotEncoder, а також оптимізації розрахунку матриць невідповідностей і побудови ROC-кривих. Мій запит: 'Напиши шаблон ColumnTransformer для scikit-learn, який масштабує числові ознаки, кодує категоріальні через OneHotEncoder (handle_unknown=ignore, sparse_output=False), відбирає топ-15 ознак за feature_importances_ RandomForestClassifier і навчає модель'.
Код я адаптував до Automobile_data.csv: обробив символи '?' у 7 стовпцях, бінаризував symboling > 0, забезпечив суворий поділ train_test_split зі stratify=y та виключив fit на тесті для захисту від data leakage. Також реалізував блок PCA з порогом 90% дисперсії.

## Висновок:

Під час виконання лабораторної роботи №4 я опанував практичні методи інженерії даних для задачі класифікації на прикладі датасету характеристик автомобілів.
Основні результати роботи:
- Досліджено структуру даних: вибірка містить 205 спостережень і 25 вхідних ознак (15 числових та 10 категоріальних). Виявлено пропуски у 7 колонках, які усунуто заповненням медіанами для числових параметрів та модою для категоріальних. Повних дублікатів записів немає. Цільову змінну symboling бінаризовано на два класи: безпечні (symboling <= 0, 44.88%) та ризикові (symboling > 0, 55.12%). Завдяки помірному балансу штучні методи на кшталт SMOTE не знадобилися.
- Пайплайн підготовки даних: застосування ColumnTransformer (StandardScaler для числових і OneHotEncoder для категорійних змінних) розширило ознаковий простір з 25 до 73 ознак за рахунок 58 бінарних фіктивних змінних. Забезпечено суворе дотримання принципу ізоляції test-вибірки (fit тільки на train) для запобігання data leakage.
- Порівняння моделей RandomForestClassifier (n_estimators=100, random_state=42): Model A (73 ознаки): Accuracy=0.8780, F1=0.8837, ROC-AUC=0.9167 (TN=17, FP=1, FN=4, TP=19); Model B (ТОП-15 ознак): Accuracy=0.8780, F1=0.8889, ROC-AUC=0.9227 (TN=16, FP=2, FN=3, TP=20); Model C (PCA 13 компонент): Accuracy=0.8049, F1=0.8182, ROC-AUC=0.9082 (TN=15, FP=3, FN=5, TP=18).
- Вплив інженерії ознак: відбір 15 найбільш релевантних факторів дозволив скоротити розмірність простору майже у 5 разів (з 73 до 15), підвищивши повноту Recall з 0.8261 до 0.8696 та F1-score з 0.8837 до 0.8889 завдяки фільтрації розрідженого шуму від One-Hot кодування.
- Оцінка доцільності PCA: застосування методу PCA виявилося неефективним для даної задачі (погіршення Accuracy на 7.31% та F1-score на 7.07%), оскільки лінійні комбінації ознак порушують ортогональність розбиттів у деревах рішень та повністю нівелюють фізичну інтерпретованість ознак, необхідну для оцінки страхових ризиків.
- Фінальний вибір: як найкращий обрано пайплайн на основі Набору B (попередня обробка + ТОП-15 ознак + Random Forest), який поєднує найвищу діагностичну якість, мінімум критичних помилок другого роду (FN=3) та повну прозорість для бізнесу.